

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

C++ 排序与自定义比较函数

Sorting in C++: Custom Comparators and Multi-Key Sorting

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 18 日

目录

1. 排序与 sort 函数
2. 自定义比较函数
3. 多关键字排序
4. pair 与 vector 的排序
5. 常见陷阱
6. 练习

目录

7 总结

目录

1. 排序与 sort 函数
2. 自定义比较函数
3. 多关键字排序
4. pair 与 vector 的排序
5. 常见陷阱
6. 练习

目录

7 总结

为什么需要排序

排序：把一组数据按照某种规则重新排列，使元素之间满足单调关系。

排序的常见用途

- 二分查找的前置条件
- 统计频次、去重
- 按关键字输出结果
- 贪心、动态规划等算法的预处理

核心问题：给定两个元素 A、B，如何判断 A 是否应当排在 B 前面？

$$\text{排序} = \text{指定规则} + \text{反复比较交换}$$

sort 函数的基本用法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

函数原型

```
1 sort(first, last);
2 sort(first, last, cmp);
```

参数说明

- first、last：迭代器，表示区间 [first, last)
- cmp：可选的比较函数，决定排序规则
- 默认不传 cmp 时按升序排列

常用调用形式

```
1 sort(a, a + n);                  // 对数组升序排序
2 sort(a + 1, a + 1 + n);          // 对下标 1 到 n 排序
3 sort(v.begin(), v.end());        // 对 vector 排序
```

注意：第二个参数是右端点的下一位置，区间左闭右开。

目录

1 排序与 sort 函数

2 自定义比较函数

3 多关键字排序

4 pair 与 vector 的排序

5 常见陷阱

6 练习

目录

7 总结

比较函数的语义

比较函数 $cmp(A, B)$ **返回一个布尔值，表示** “A **是否严格排在** B **前面**”。

语义约定

- **返回** true：A **应当排在** B **前面**
- **返回** false：A **不排在** B **前面**
- **严格弱序**：$cmp(A, A)$ **必须为** false

两种常见视角

- **升序**：A **更小则返回** true
- **降序**：A **更大则返回** true

比较函数只需回答：谁在前

整数降序排序

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

比较函数

```
1 bool cmp1(const int &A, const int &B)
2 {
3     return A > B;   // 降序
4 }
```

调用

```
1 int a[Maxn + 5];
2 sort(a + 1, a + 1 + n, cmp1);
```

结果分析

- 传入 $A = 3$、$B = 5$：$3 > 5$ 为 false，不交换
- 传入 $A = 5$、$B = 3$：$5 > 3$ 为 true，A 在前
- 排序后数组从大到小排列

目录

1 排序与 sort 函数

2 自定义比较函数

3 多关键字排序

4 pair 与 vector 的排序

5 常见陷阱

6 练习

目录

7 总结

结构体的多关键字排序

结构体定义

```
1 struct node
2 {
3     int x, y;
4 } b[Maxn + 5];
```

排序需求：先按 x 升序，x 相同时按 y 升序。

比较函数

```
1 bool cmp2(const node &A, const node &B)
2 {
3     if (A.x != B.x)
4         return A.x < B.x;
5     else
6         return A.y < B.y;
7 }
```

多关键字排序的写法要点

分层次比较

- 第一关键字不同时，直接返回第一关键字的比较结果
- 第一关键字相同时，退到第二关键字继续比较
- 更多关键字依次嵌套即可

调用方式

```
sort(b + 1, b + 1 + m, cmp2);
```

等价写法

```
bool cmp2(const node &A, const node &B)
{
    if (A.x != B.x) return A.x < B.x;
    return A.y < B.y;
}
```

关键点：else 分支可以省略，因为 if 已处理了 A.x 不等于 B.x 的情形。

目录

1 排序与 sort 函数

2 自定义比较函数

3 多关键字排序

4 pair 与 vector 的排序

5 常见陷阱

6 练习

目录

7 总结

pair 的默认排序规则

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

pair 的定义

```
pair<int, double> c[Maxn + 5];
// pair 相当于 {first, second}
```

默认比较优先级

- 第一关键字：first
- 第二关键字：second
- 依次按字典序比较

直接排序

```
sort(c, c + k);   // 无需自定义 cmp
```

适用场景：二元组排序、图论的边排序、事件点排序。

vector 与排序

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

声明与初始化

```
1 vector<int> v;
2 v.push_back(10);
3 v.push_back(20);
4 v.push_back(30);
5 v.push_back(40);
6 v.push_back(50);
```

随机访问

```
1 cout << v[3] << endl;   // 输出 40
```

排序

```
1 sort(v.begin(), v.end());
2 sort(v.begin(), v.end(), greater<int>());
```

关键点：vector 的迭代器是 begin() 与 end()，同样遵循左闭右开。

目录

1 排序与 sort 函数

2 自定义比较函数

3 多关键字排序

4 pair 与 vector 的排序

5 常见陷阱

6 练习

目录

7 总结

比较函数的常见错误

错误一：比较函数不严格

- 写成小于等于：$A \leq B$ 会让 $cmp(A, A)$ 为 true，破坏严格弱序
- 后果：sort 内部行为未定义，可能死循环或乱序

错误二：比较逻辑颠倒

- 想要降序却写成小于号，结果仍是升序

错误三：多关键字漏分支

- 只比较第一关键字，忽略了相同值时的第二关键字

比较函数必须严格弱序，且与排序目标一致

区间与下标错误

错误一：区间写错

```
1 sort(a, a + n);              // 对 a[0..n-1] 排序
2 sort(a + 1, a + 1 + n);      // 对 a[1..n] 排序
```

错误二：忘记加一

```
1 // 下标从 1 开始时，错误写法
2 sort(a + 1, a + n);          // 漏了 a[n]
```

错误三：vector 用错迭代器

```
1 sort(v, v + n);              // 错误：vector 不是指针
2 sort(v.begin(), v.end());    // 正确
```

记忆口诀：左闭右开，末位加一。

目录

1 排序与 sort 函数

2 自定义比较函数

3 多关键字排序

4 pair 与 vector 的排序

5 常见陷阱

6 练习

目录

7 总结

练习：学生成绩排序

题目：给出 n 个学生，每个学生有学号、总分。要求按总分从高到低排序，总分相同时按学号从小到大排序。

示例输入

```
3
1001 285
1002 290
1003 285
```

示例输出

```
1002 290
1001 285
1003 285
```

练习：解法

四步法

- 结构：定义结构体存储学号和总分
- 第一关键字：总分降序
- 第二关键字：学号升序
- 调用：sort 传入自定义 cmp

参考代码

练习：解法


```cpp
struct stu {
    int id, score;
} s[Maxn + 5];

bool cmp(const stu &A, const stu &B) {
    if (A.score != B.score)
        return A.score > B.score;
    return A.id < B.id;
}

sort(s + 1, s + 1 + n, cmp);
```

为什么这样转移：先按分数排出高低，分数相同的两名学生再用学号区分先后，形成两级比较。

目录

1. 排序与 sort 函数
2. 自定义比较函数
3. 多关键字排序
4. pair 与 vector 的排序
5. 常见陷阱
6. 练习

目录

⑦ 总结

总结：排序的三层抽象

1. **基础调用**：sort(first, last) 按升序排，第二个参数是右端点的下一位置
2. **自定义规则**：传入 cmp 函数，返回 true 表示 A 应排在 B 前面
3. **多关键字**：先比第一关键字，相同再比第二关键字，依次嵌套

使用要点

- 比较函数必须严格弱序，$cmp(A, A)$ 为 false
- pair 自带字典序，无需自定义 cmp
- 区间左闭右开，sort(a, a + n) 排 a[0..n-1]

排好一个序列 = 说清楚谁在前

附录：完整示例代码


```cpp
#include <bits/stdc++.h>
#define Maxn 100000
using namespace std;
int a[Maxn + 5];

struct node {
    int x, y;
} b[Maxn + 5];

bool cmp1(const int &A, const int &B) {
    return A > B;   // 降序
}

bool cmp2(const node &A, const node &B) {
    if (A.x != B.x) return A.x < B.x;
    return A.y < B.y;
}

```

附录：完整示例代码


```
19 int main() {
20     int n;
21     cin >> n;
22     for (int i = 1; i <= n; i++)
23         scanf("%d", &a[i]);
24     sort(a + 1, a + 1 + n, cmp1);
25
26     int m;
27     cin >> m;
28     for (int i = 1; i <= m; i++)
29         scanf("%d%d", &b[i].x, &b[i].y);
30     sort(b + 1, b + 1 + m, cmp2);
31
32     return 0;
33 }
```

附录：常见陷阱速查

错误	修复
比较函数用 $\leq$ 或 $\geq$	改用严格的 $<$ 或 $>$
多关键字漏第二关键字	第一关键字相等时继续比第二关键字
sort 第二个参数漏加一	下标 1 到 n 应写 a + 1 + n
vector 用指针排序	改用 v.begin() 与 v.end()

严格弱序 + 左闭右开 + 分层次比较